

Applying Encapsulation and Abstraction in Online Banking

An OOAD perspective for understanding how banking systems are designed using core object-oriented principles



What is OOAD?

Object Oriented Analysis (OOA)

Identifying real-world entities (objects) in a system —
e.g., **Customer**, **Account**, **Transaction**

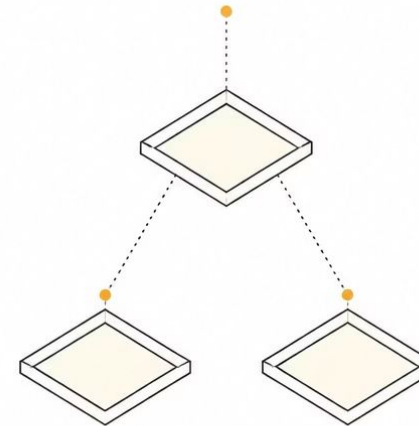
Object Oriented Design (OOD)

Structuring those objects with attributes, behaviors,
and relationships to build a working system

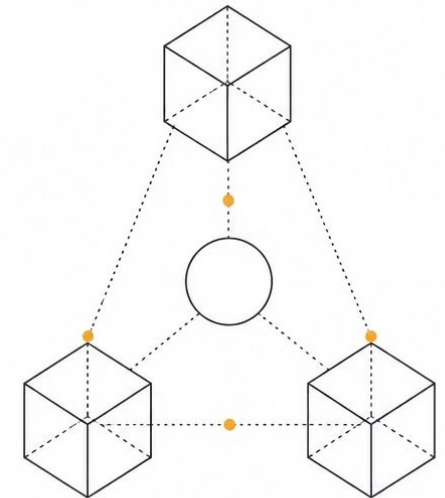
The Bridge

OOAD bridges the gap between real-world banking
problems and software solutions

Phase 1 - Analysis
(identify Customer, Account,
Transaction objects from real world)



Phase 2 - Design
(define attributes, methods,
relationships to build the system)



Key Principles: Encapsulation & Abstraction

Encapsulation

Bundling data (attributes) and methods (behaviors) together, and **restricting direct access** to sensitive data

Abstraction

Hiding complex internal logic and **exposing only what the user needs** to interact with

📌 Together, these principles make banking systems **secure, modular, and easy to maintain**.

Encapsulation in Online Banking



How Encapsulation Works in Practice

→ Account Balance

Stored as a **private attribute** — users can only view or update it through controlled methods like `deposit()` or `withdraw()`

→ PIN / Password

Never exposed directly; accessed only through `authenticate()` method with validation logic built in

→ Why It Matters

Prevents **unauthorized access** and accidental data corruption across the system

Abstraction in Online Banking

User Interface Layer

Customers click "**Transfer Funds**" — they don't see the validation, ledger updates, or fraud checks happening behind the scenes

Abstract Classes & Interfaces

A generic `BankAccount` class defines common behavior; `SavingsAccount` and `CurrentAccount` implement their own rules

Simplifies Complexity

Reduces cognitive load for both **end users** and **developers** maintaining the system



Real-World Banking Use Cases



Fund Transfer

`transfer()` method encapsulates debit, credit, and logging — the caller only provides **amount and recipient**



Loan Processing

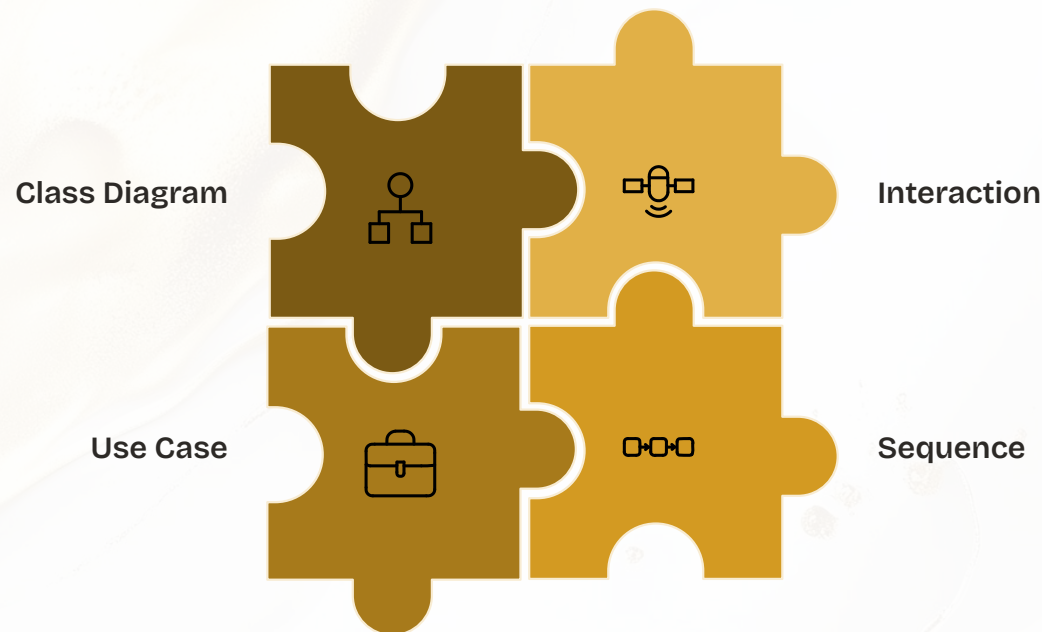
Abstract `LoanAccount` class hides EMI calculation logic; subclasses handle **Home Loan** vs. **Personal Loan** differently



Authentication

Login system abstracts **OTP generation**, session management, and token validation from the end user entirely

OOAD Modeling: Key Diagrams



1

Class Diagram

Shows objects like Customer, Account, Transaction with **private/public** attributes and methods

2

Use Case Diagram

Captures interactions — Customer **logs in**, checks balance, transfers funds as distinct system use cases

3

Sequence Diagram

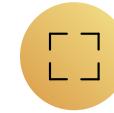
Illustrates how objects **communicate** and pass messages during a complete transaction flow

Why OOAD Matters in Banking Systems



Security

Encapsulation ensures sensitive data like **balance and credentials** are never directly accessible from outside the object



Scalability

Abstraction allows **new account types or features** to be added without breaking existing functionality



Maintainability

Clear object boundaries make **debugging and updates** faster, safer, and less error-prone

